1. Software Design Specification

FUNCTIONAL

Document Control	
Author(s)	Michael Gardner
Document Version	4.1.0
Applies to FUNCTIONAL	^4.1
Status	Released
Last Updated	2026-04-27
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2026 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	library
Library Role	utility
Application Role	N/A
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Software Design Specification ... 1
2. Introduction ... 1
   - 2.1. Purpose ... 1
   - 2.2. Scope ... 1
   - 2.3. References ... 1
3. Architectural Overview ... 1
   - 3.1. Architecture Style ... 1
   - 3.2. Dependency Rules ... 1
   - 3.3. SPARK Boundary ... 2
4. Package Structure ... 2
   - 4.1. Directory Layout ... 2
   - 4.2. Package Descriptions ... 2
5. Type Definitions ... 3
   - 5.1. Result Type ... 3
   - 5.2. Option Type ... 3
   - 5.3. Either Type ... 3
6. Design Patterns ... 4
   - 6.1. Discriminated Union Pattern ... 4
   - 6.2. Generic Instantiation Pattern ... 4
   - 6.3. Railway-Oriented Programming ... 4
   - 6.4. Declarative Exception Mapping ... 4
   - 6.5. RAII Guard Pattern ... 4
7. Error Handling Strategy ... 5
   - 7.1. Core Principle ... 5
   - 7.2. Module Responsibilities ... 5
   - 7.3. Try Usage Guidelines ... 5
8. Concurrency Design ... 5
   - 8.1. Thread Safety ... 5
   - 8.2. Future Concurrency Support ... 5
9. Performance and Memory Design ... 5
   - 9.1. Zero-Overhead Abstractions ... 5
   - 9.2. Memory Management ... 5
10. Build and Deployment ... 6
    - 10.1. Project Structure ... 6
    - 10.2. Build Profiles ... 6
    - 10.3. Platform Support ... 6
11. SPARK Readiness Assessment ... 6
    - 11.1. Overview ... 6
    - 11.2. Module Assessment ... 6
    - 11.3. SPARK-Compatible Components ... 6
12. Testing Strategy ... 6
    - 12.1. Test Organization ... 6
    - 12.2. Testing Approach ... 7
13. Design Decisions ... 7
    - 13.1. No Tagged Types in Core ... 7
    - 13.2. Deprecation of Procedural Try Functions ... 7
    - 13.3. Exception Suppression in Scoped ... 7

14. Appendices 8
14.1. Change History 8

2. Introduction

2.1. Purpose

This Software Design Specification (SDS) describes the architecture, package structure, type definitions, design patterns, and implementation decisions for the **Functional** library.

2.2. Scope

This document covers:

- package hierarchy and dependency rules,
- SPARK boundary design,
- core type definitions and their rationale,
- design patterns employed (discriminated unions, generic instantiation, railway-oriented programming, declarative exception mapping, RAII guards),
- error handling strategy, and
- build configuration.

2.3. References

- Software Requirements Specification (SRS).
- Ada 2022 Language Reference Manual (ISO/IEC 8652:2023).
- SPARK 2014 Reference Manual.

3. Architectural Overview

3.1. Architecture Style

Functional is a flat utility library, not a layered DDD/Clean/Hexagonal project. It is organized by module rather than by architectural layer. The primary structural concern is the **SPARK boundary**: core types (Result, Option, Either) are SPARK-compatible, while exception-handling infrastructure (Try, Scoped) is not.

Module Family	Purpose	SPARK_Mode
Result, Option, Either	Pure functional types — discriminated unions with combinator operations.	On
Try	Exception-to-Result/Option bridges for architectural boundaries.	Off
Scoped	RAII guards using Ada.Finalization.	Off
Version	Library version constants.	On

3.2. Dependency Rules

Package	Dependencies	SPARK_Mode
Functional	None	On
Functional.Either	Functional	On
Functional.Option	Functional	On
Functional.Result	Functional	On
Functional.Scoped	Ada.Finalization	Off
Functional.Try	Ada.Exceptions, Result, Option	Off
Functional.Try.Map_To_Result	Try	Off

Package	Dependencies	SPARK_Mode
Functional.Try.Map_To_Result_With_Param	Try	Off
Functional.Try.To_Option	Try, Option	Off
Functional.Try.To_Result	Try, Result	Off
Functional.Version	None	On

3.3. SPARK Boundary

- **SPARK_Mode On**: Result, Option, Either (pure functional types with no side effects).
- **SPARK_Mode Off**: Try (exception handling), Scoped (finalization).

This boundary ensures core types are formally verifiable while allowing exception handling at system boundaries.

4. Package Structure

4.1. Directory Layout

```
src/
├── functional.ads                                # Root package
├── functional-result.ads / .adb                  # Result type
├── functional-option.ads / .adb                  # Option type
├── functional-either.ads / .adb                  # Either type
├── functional-try.ads / .adb                     # Try exception bridges
├── functional-try-map_to_result.ads / .adb       # Declarative mapping
├── functional-try-map_to_result_with_param.ads / .adb
├── functional-try-to_result.ads / .adb           # Procedural mapping
├── functional-try-to_option.ads / .adb           # Exception-to-Option
├── functional-scoped.ads / .adb                  # RAII guards
└── version/
    └── functional-version.ads                    # Version info
```

4.2. Package Descriptions

Package	Purpose	SPARK
`Functional`	Root namespace, common declarations.	On
`Functional.Either`	Left/right discriminated union with combinators.	On
`Functional.Option`	Presence/absence discriminated union with combinators.	On
`Functional.Result`	Success/failure discriminated union with combinators.	On
`Functional.Scoped`	RAII guard generics.	Off
`Functional.Try`	Exception-to-Result/Option bridge infrastructure.	Off
`Functional.Try.Map_To_Result`	Declarative exception mapping (no extra parameter).	Off
`Functional.Try.Map_To_Result_With_Param`	Declarative exception mapping (parameterized action).	Off
`Functional.Try.To_Option`	Exception-to-Option for probe operations.	Off

Package	Purpose	SPARK
`Functional.Try.To_Result`	Procedural exception mapping (legacy).	Off
`Functional.Version`	Library version constants.	On

5. Type Definitions

5.1. Result Type

```
generic
   type T is private;
   type E is private;
package Functional.Result with SPARK_Mode => On is
   type Result (Is_Ok : Boolean := False) is record
      case Is_Ok is
         when True  => Value : T;
         when False => Error : E;
      end case;
   end record;
end Functional.Result;
```

Design Rationale:

- Discriminated union ensures exactly one of Value or Error is present.
- Default discriminant `False` allows uninitialized declarations.
- Generic parameters allow any success/error type pair.
- Zero heap allocation.

5.2. Option Type

```
generic
   type T is private;
package Functional.Option with SPARK_Mode => On is
   type Option (Is_Some : Boolean := False) is record
      case Is_Some is
         when True  => Value : T;
         when False => null;
      end case;
   end record;
end Functional.Option;
```

Design Rationale:

- Discriminated union eliminates null references.
- Preelaborate-safe for domain layer usage.
- Zero heap allocation.

5.3. Either Type

```
generic
   type L is private;
   type R is private;
package Functional.Either with SPARK_Mode => On is
   type Either (Is_Left : Boolean := True) is record
      case Is_Left is
         when True  => Left_Value  : L;
         when False => Right_Value : R;
      end case;
```

```
   end record;
end Functional.Either;
```

Design Rationale:
- Neither side is designated as "error" (distinguishes Either from Result).
- Useful for union types, parsing, and validation branches.

6. Design Patterns

6.1. Discriminated Union Pattern

All core types use Ada discriminated records:
- Compile-time enforcement of valid access via discriminant checks.
- Zero heap allocation.
- SPARK compatible.

6.2. Generic Instantiation Pattern

```
--  Consumer instantiates with their own types
package User_Result is new Functional.Result
  (T => User_Type, E => Error_Kind);
```

Benefits:
- Type safety (different instantiations cannot be mixed).
- Monomorphization (no runtime dispatch).
- Clear at call site what types are involved.

6.3. Railway-Oriented Programming

Operations chain through Map / And_Then / Or_Else:

```
Result := User_Result.Ok (User)
  .Map (Validate'Access)
  .And_Then (Save'Access)
  .Or_Else (Log_And_Default'Access);
```

Error propagation is automatic — no explicit checking between operations.

6.4. Declarative Exception Mapping

Map_To_Result uses data (mapping arrays) instead of procedural code:

```
Mappings : constant Try_Read.Mapping_Array :=
  [(Name_Error'Identity,  Not_Found),
   (Use_Error'Identity,   Permission_Error)];

Result := Try_Read.Run (Path, Mappings);
```

Benefits:
- Self-documenting (exception table is visible data).
- No procedural Map_Exception boilerplate.
- Easy to add or remove mappings.

6.5. RAII Guard Pattern

Scoped uses Ada finalization for automatic cleanup:

```
declare
   Guard : File_Guard.Guard (File'Access);
begin
```

```
   --  Work with file
end;  --  Guard.Finalize automatically closes file
```

7. Error Handling Strategy

7.1. Core Principle

Errors are values, not exceptions.

Consumers of the Functional library use Result and Option types for all expected failure paths. Exceptions are caught at boundaries (the consuming project's infrastructure layer) using Try and converted to Result.

7.2. Module Responsibilities

Module	Error Handling Role
Either	Carry left/right values through combinators.
Option	Carry present/absent values through combinators.
Result	Carry success/failure values through combinators.
Scoped	Suppress exceptions from cleanup actions to prevent double-exception scenarios.
Try	Catch exceptions at boundaries and convert to Result or Option.

7.3. Try Usage Guidelines

Scenario	Use
Multiple exception types to multiple errors	Map_To_Result_With_Param
Catch-all with default error	Map_To_Result.Run_Catch_All
Probe/check operations	To_Option + Unwrap_Or
Legacy procedural mapping	To_Result

8. Concurrency Design

8.1. Thread Safety

Functional is a sequential library. All types are value types (discriminated records) and can be safely used in concurrent contexts without synchronization, provided each task operates on its own instances.

8.2. Future Concurrency Support

Concurrency is out of scope for the current design. The value-type nature of all core types means no design changes are anticipated for concurrent use.

9. Performance and Memory Design

9.1. Zero-Overhead Abstractions

- All core types are stack-allocated discriminated records.
- Static dispatch via generics (no tagged types, no dynamic dispatch).
- No runtime type information overhead.

9.2. Memory Management

- No heap allocation in any module.
- All types are bounded by their discriminant and generic parameters.

- Suitable for embedded and memory-constrained environments.

10. Build and Deployment

10.1. Project Structure

Project	Purpose
`functional.gpr`	Main library build.
`test/functional_test.gpr`	Test suite build.

10.2. Build Profiles

Profile	Optimization	Assertions	Debug	Use Case
Development	-O0	Enabled	-g	Daily development.
Validation	-O1	Enabled	-g	Pre-release testing.
Release	-O2	Disabled	None	Production.

10.3. Platform Support

The library supports all platforms and processor architectures listed in the project profile. No platform-specific code exists in the library.

11. SPARK Readiness Assessment

11.1. Overview

Functional targets a **spark-targeted** assurance posture. Core types (Result, Option, Either) are fully SPARK-compatible. Infrastructure modules (Try, Scoped) are outside SPARK due to their use of Ada.Exceptions and Ada.Finalization.

11.2. Module Assessment

Module	SPARK	Notes
Either	On	Pure functional type, fully provable.
Option	On	Pure functional type, fully provable.
Result	On	Pure functional type, fully provable.
Scoped	Off	Ada.Finalization is outside SPARK.
Try	Off	Exception handling is inherently outside SPARK.
Version	On	Constants only.

11.3. SPARK-Compatible Components

All operations on Result, Option, and Either (constructors, discriminant checks, Map, And_Then, Or_Else, Unwrap_Or, Match, Fold) are SPARK-compatible and provable.

12. Testing Strategy

12.1. Test Organization

Tests are organized by module in `test/unit/` with a shared framework in `test/common/`. SPARK legality tests are in `test/spark/`.

12.2. Testing Approach

- **Unit tests**: Exercise all public operations for each module.
- **SPARK tests**: Verify that core types can be instantiated under SPARK_Mode On.
- **Integration tests**: Verify Try with real exception scenarios.

13. Design Decisions

13.1. No Tagged Types in Core

Decision: Core types (Result, Option, Either) are not tagged.

Rationale:

- Static dispatch eliminates runtime overhead.
- SPARK compatibility (tagged types have limitations in SPARK).
- Generic instantiation provides the needed polymorphism.

13.2. Deprecation of Procedural Try Functions

Decision: Deprecate Try_To_Result in favor of Map_To_Result.

Rationale:

- Declarative mappings are more maintainable than procedural Map_Exception functions.
- Exception tables are self-documenting.

Exception: Option-based Try functions (To_Option) are NOT deprecated — they serve valid use cases for probe operations.

13.3. Exception Suppression in Scoped

Decision: Scoped guards catch and suppress exceptions from Release.

Rationale:

- Prevents double-exception scenarios.
- Ensures cleanup completes even if Release fails.
- Consistent with RAII semantics in other languages.

14. Appendices

14.1. Change History

Version	Date	Author	Changes
4.1.0	2025-12-18	Michael Gardner	Deprecate procedural Try functions; keep Option-based Try functions. Migrate to Typst format.
4.0.0	2025-12-12	Michael Gardner	Remove Expect functions; add Map_To_Result packages.
3.0.0	2025-11-30	Michael Gardner	Initial architecture with Result, Option, Either, Try, Scoped.